

RetinAnalysis Setup — Mac

Everything runs in **Terminal**.

Skip a step if you already have that tool installed. Each step says what it's for so you can tell.

1. Install Git and a C compiler

```
xcode-select --install
```

This one command installs both Git (for cloning the repo) and `clang` (the C/C++ compiler needed later to build a submodule) — no separate downloads needed.

2. Install Docker Desktop

Runs the local MySQL/DataJoint database that stores experiment metadata. Download the build that matches your chip (Apple Silicon or Intel):

<https://docs.docker.com/desktop/>

Launch it once after installing, then leave it open whenever you need database access (queries, `ra.populate_database()`). Just `import retinanalysis` does not need it running.

3. Confirm you have conda

```
conda --version
```

If that fails, install Miniconda first: <https://docs.conda.io/en/latest/miniconda.html>

4. Clone the repository

`--recursive` is required — it pulls in a submodule (`vision-utils`) that the package needs to read Vision's data files.

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
```

If you forget `--recursive`, fix it afterward with `git submodule update --init --recursive`.

5. Create the environment and install

```
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install lib/artificial-retina-software-pipeline/utilities/
```

The first `pip install` installs retinanalysis itself and everything it depends on (including `opencv-python` — you never need to install OpenCV separately). The second installs the submodule from step 4 — this is the step that needs the C compiler from step 1.

6. Set up config.ini

This file tells the package where your data actually lives on this machine. It's not tracked in git (everyone's paths are different), but a template with the right structure is:

```
cp src/retinanalysis/config/config.ini.example src/retinanalysis/config/config.ini
```

Open `src/retinanalysis/config/config.ini` and edit the `[DEFAULT]` section with real paths, e.g.:

```
[DEFAULT]
analysis = /Volumes/Array-data/sorted
data = /Volumes/Array-data/sorted
raw = /Volumes/Array-data/raw
h5 = /Volumes/Array-data/h5
meta = /Volumes/Array-data/dj_meta
tags = /Volumes/Array-data/tags
query = /Volumes/Array-data/sorted
user = your_username
```

`/Volumes/Array-data` is wherever your shared network drive is mounted on this Mac — check Finder → Go → Network (the exact name may differ). `user` is your own username. You only need to fill in

[DEFAULT] (and [SECONDARY] only if you actually have a second data location) — leave the other sections alone.

7. Start the local database

Docker Desktop must be open for this to work.

```
mkdir -p ../retinanalysis-database
cp docker-compose.yaml ../retinanalysis-database/
cd ../retinanalysis-database
docker compose up -d
cd ../retinanalysis
```

8. Install Jupyter and launch it

```
pip install jupyterlab
jupyter lab
```

This opens a browser tab. Because Jupyter was installed inside the same `retinanalysis` environment you just activated, it's automatically using the right Python — no separate kernel setup needed.

9. Populate the database

Open any notebook in `demos/` and run in a cell:

```
import retinanalysis as ra
ra.populate_database()
```

Quick reference — every command in order

```
xcode-select --install
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install lib/artificial-retina-software-pipeline/utilities/
cp src/retinanalysis/config/config.ini.example src/retinanalysis/config/config.ini
# edit config.ini with real paths, then:
mkdir -p ../retinanalysis-database
cp docker-compose.yaml ../retinanalysis-database/
cd ../retinanalysis-database
docker compose up -d
cd ../retinanalysis
pip install jupyterlab
jupyter lab
```

Troubleshooting

ModuleNotFoundError: No module named 'cv2' on the very first `import retinanalysis` cell

Not a missing OpenCV install — `opencv-python` is already installed automatically by step 5. This means Jupyter is running a different Python than the `retinanalysis` environment. Close Jupyter, run `conda activate retinanalysis`, then launch `jupyter lab` from that same terminal. Once the notebook is open, check the kernel name in the top-right corner — if it's not `retinanalysis`, switch it (or if it's missing from the list, run `pip install ipykernel` then `python -m ipykernel install --user --name retinanalysis` and refresh).

Installing the `lib/.../utilities/` submodule fails with a compiler error

`xcode-select --install` from step 1 didn't finish. Run it again and wait for it to complete before retrying.

Database calls fail / connection refused

Docker Desktop isn't open, or the container isn't running. Open Docker Desktop and confirm the `retinanalysis-database` container shows a stop icon (running), not a play icon.

Cloned without `--recursive`

Run `git submodule update --init --recursive`, then redo step 5.